

Morphic Studio Software Architecture Specification

Document 38 — Monorepo & Repository Architecture

Version: 1.0

Status: Approved

1. Purpose

This document defines the official repository structure for Morphic Studio.

It establishes how source code, shared libraries, services, infrastructure, documentation, tests, assets, and future applications are organized.

The objective is to ensure the codebase remains understandable, scalable, AI-friendly, and maintainable from Version 1 through future enterprise editions.

This structure is mandatory unless superseded by a future architecture decision record (ADR).

2. Architectural Principles

The repository follows these core principles.

2.1 Single Source Repository

All official Morphic Studio source code resides within a single Git repository.

Benefits include:

- Unified version control
 - Shared packages
 - Shared types
 - Shared tooling
 - Simplified CI/CD
 - Better AI coding assistance
 - Easier dependency management
-

2.2 Domain-Driven Organization

The repository is organized around business domains rather than technical layers.

Examples of domains include:

- Story
- Characters
- Worlds
- Assets
- Production
- Publishing
- AI
- Users
- Authentication

Each domain owns its own business logic.

2.3 Modular Design

Every subsystem must be independently maintainable.

Modules should expose clear interfaces and minimize internal knowledge of other modules.

2.4 Shared First

Reusable code belongs in shared packages.

Examples:

- Design System
- UI Components
- Authentication
- Validation
- Shared Types
- API Client
- Logging
- Configuration

Duplicate implementations are discouraged.

2.5 Replaceable Infrastructure

Infrastructure choices must remain replaceable.

Changing storage providers, AI vendors, databases, or deployment targets should not require rewriting application logic.

3. Repository Layout

```
morphic-studio/  
  
apps/  
  web/  
  api/  
  admin/  
  docs/  
  desktop/      (future)  
  mobile/       (future)  
  
packages/  
  ui/  
  design-system/  
  icons/  
  themes/  
  auth/  
  api-client/  
  shared-types/  
  validation/  
  configuration/  
  logger/  
  utilities/  
  
domains/  
  story/  
  characters/  
  worlds/  
  assets/  
  production/  
  publishing/  
  projects/  
  users/  
  notifications/  
  
services/
```

- ai-gateway/
- story-bible/
- creative-memory/
- planner/
- render/
- search/
- export/
- analytics/
- database/
 - schema/
 - migrations/
 - seeds/
 - backups/
 - documentation/
- infrastructure/
 - docker/
 - deployment/
 - monitoring/
 - observability/
 - scripts/
 - security/
- documentation/
 - architecture/
 - api/
 - frontend/
 - backend/
 - ai/
 - testing/
 - deployment/
 - adr/
- tests/
 - unit/
 - integration/
 - end-to-end/
 - performance/
 - accessibility/
 - security/
- assets/
 - branding/
 - icons/
 - fonts/
 - illustrations/
 - mockups/

```
samples/

.github/
  workflows/
  issue_templates/
  pull_request_templates/

README.md
LICENSE
CONTRIBUTING.md
CODE_OF_CONDUCT.md
SECURITY.md
CHANGELOG.md
pnpm-workspace.yaml
turbo.json
package.json
```

4. Domain Responsibilities

Every domain owns its business rules.

Story Domain

Responsible for:

- Chapters
- Scenes
- Episodes
- Story Structure
- Story Metadata

Character Domain

Responsible for:

- Character Profiles
 - Relationships
 - Character Development
 - Voice Profiles
 - Character Consistency
-

World Domain

Responsible for:

- Locations
 - Cultures
 - Governments
 - Magic Systems
 - Technology Levels
 - Geography
 - Lore
-

Asset Domain

Responsible for:

- Images
 - Audio
 - Video
 - Models
 - Asset Metadata
 - Asset Versioning
 - Asset Search
-

Production Domain

Responsible for:

- Storyboards
 - Comics
 - Motion Comics
 - Animation
 - Rendering
 - Production Timeline
-

Publishing Domain

Responsible for:

- Export
- Publishing
- Distribution

- Version History
 - Publishing Targets
-

5. Shared Packages

Shared packages must contain no business-specific logic.

Examples include:

- Buttons
 - Cards
 - Icons
 - Typography
 - Theme Engine
 - API Client
 - Validation
 - Authentication Helpers
-

6. Service Layer

Services contain cross-domain capabilities.

Examples:

AI Gateway

Creative Memory

Story Bible

Search

Analytics

Notification

Render Queue

These services communicate through APIs and events rather than direct coupling.

7. Dependency Rules

Allowed:

Applications → Domains

Applications → Shared Packages

Domains → Shared Packages

Domains → Services

Forbidden:

Shared Packages → Applications

Shared Packages → Domains

Circular Dependencies

Business Logic inside UI Components

8. Naming Standards

All folders use:

- lowercase
- kebab-case
- descriptive names

Avoid abbreviations unless universally understood.

Examples:

creative-memory

story-bible

character-service

render-queue

9. Repository Governance

Every major architectural change requires an Architecture Decision Record (ADR).

ADRs should include:

- Context
- Decision
- Alternatives
- Consequences

This creates a historical record of architectural evolution.

10. Quality Standards

The repository must support:

- Static Analysis
- Type Safety
- Automated Formatting
- Linting
- Unit Testing
- Integration Testing
- End-to-End Testing
- Accessibility Testing
- Security Scanning

Quality gates are enforced before merging code into the main branch.

11. Future Expansion

The repository is intentionally designed to accommodate:

- Mobile applications
- Desktop applications
- Enterprise Edition
- Plugin Marketplace
- Community Extensions
- AI Provider Marketplace
- Team Collaboration Features
- Education Edition
- API SDKs

- Public Integrations

No future expansion should require restructuring the repository.

12. Acceptance Criteria

This architecture is considered successful if:

- New developers understand the repository quickly.
 - AI coding agents can navigate the project without ambiguity.
 - Business logic remains isolated from infrastructure.
 - Modules are independently testable.
 - Features can be added without large-scale refactoring.
 - The codebase remains maintainable as Morphe Studio grows.
-

Conclusion

The Morphe Studio repository architecture establishes a modular, domain-driven foundation capable of supporting a modern AI-native creative platform. It prioritizes clarity, scalability, maintainability, and long-term evolution while remaining accessible to both human developers and AI coding agents.